

1.



The screenshot displays the SIMATS Online Programming Lab interface. The top header includes the lab name and a logo. On the left, a sidebar lists various programming tasks. The main area features a C++ code editor with syntax highlighting. The code defines a `Matrix` struct and a `Matrix` class, implementing matrix operations like addition, subtraction, multiplication, and division. The `main` function demonstrates these operations with two 3x3 matrices. A console window on the right shows the output of the program, displaying the results of the matrix operations.

```
#include <iostream>
using namespace std;

struct Matrix {
    int rows, cols;
    int** data;
};

Matrix* createMatrix(int rows, int cols) {
    Matrix* mat = new Matrix();
    mat->rows = rows;
    mat->cols = cols;
    mat->data = new int*[rows];
    for (int i = 0; i < rows; i++) {
        mat->data[i] = new int[cols];
    }
    return mat;
};

void displayMatrix(Matrix* mat) {
    for (int i = 0; i < mat->rows; i++) {
        for (int j = 0; j < mat->cols; j++) {
            cout << mat->data[i][j] << " ";
        }
        cout << endl;
    }
};

Matrix* addMatrix(Matrix* mat1, Matrix* mat2) {
    Matrix* result = createMatrix(mat1->rows, mat1->cols);
    for (int i = 0; i < result->rows; i++) {
        for (int j = 0; j < result->cols; j++) {
            result->data[i][j] = mat1->data[i][j] + mat2->data[i][j];
        }
    }
    return result;
};

Matrix* subtractMatrix(Matrix* mat1, Matrix* mat2) {
    Matrix* result = createMatrix(mat1->rows, mat1->cols);
    for (int i = 0; i < result->rows; i++) {
        for (int j = 0; j < result->cols; j++) {
            result->data[i][j] = mat1->data[i][j] - mat2->data[i][j];
        }
    }
    return result;
};

Matrix* multiplyMatrix(Matrix* mat1, Matrix* mat2) {
    Matrix* result = createMatrix(mat1->rows, mat2->cols);
    for (int i = 0; i < result->rows; i++) {
        for (int j = 0; j < result->cols; j++) {
            result->data[i][j] = 0;
            for (int k = 0; k < mat1->cols; k++) {
                result->data[i][j] += mat1->data[i][k] * mat2->data[k][j];
            }
        }
    }
    return result;
};

Matrix* divideMatrix(Matrix* mat1, Matrix* mat2) {
    Matrix* result = createMatrix(mat1->rows, mat1->cols);
    for (int i = 0; i < result->rows; i++) {
        for (int j = 0; j < result->cols; j++) {
            result->data[i][j] = mat1->data[i][j] / mat2->data[i][j];
        }
    }
    return result;
};

int main() {
    Matrix* mat1 = createMatrix(3, 3);
    Matrix* mat2 = createMatrix(3, 3);
    // Initialize mat1
    mat1->data[0][0] = 1; mat1->data[0][1] = 2; mat1->data[0][2] = 3;
    mat1->data[1][0] = 4; mat1->data[1][1] = 5; mat1->data[1][2] = 6;
    mat1->data[2][0] = 7; mat1->data[2][1] = 8; mat1->data[2][2] = 9;
    // Initialize mat2
    mat2->data[0][0] = 2; mat2->data[0][1] = 3; mat2->data[0][2] = 4;
    mat2->data[1][0] = 5; mat2->data[1][1] = 6; mat2->data[1][2] = 7;
    mat2->data[2][0] = 8; mat2->data[2][1] = 9; mat2->data[2][2] = 10;
    // Perform operations
    Matrix* sum = addMatrix(mat1, mat2);
    Matrix* diff = subtractMatrix(mat1, mat2);
    Matrix* prod = multiplyMatrix(mat1, mat2);
    Matrix* div = divideMatrix(mat1, mat2);
    // Display results
    displayMatrix(sum);
    displayMatrix(diff);
    displayMatrix(prod);
    displayMatrix(div);
    return 0;
}
```

Output:

```
6 5 7
3 1 -1
15 17 19
1 0.5 0.75
0.5 0.833333 0.857143
14 12 12
0.5 0.333333 0.25
```

2.

New Code

```

1  import javax.swing.*;
2  public class Main {
3      public static void main(String[] args) {
4          JFrame frame = new JFrame("Main");
5          JPanel panel = new JPanel();
6          JLabel label = new JLabel("Hello, World!");
7          JButton button = new JButton("Click");
8          frame.add(panel);
9          panel.add(label);
10         panel.add(button);
11         frame.setVisible(true);
12         frame.setSize(300, 200);
13         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14         frame.setLocationRelativeTo(null);
15         frame.setVisible(true);
16     }
17 }

```

Input

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

Output

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17